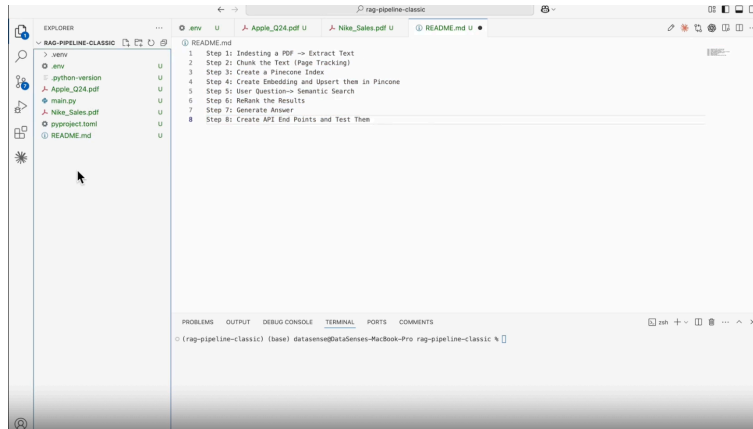


1. RAG Pipeline Architecture and Project Structure



Understanding the Pipeline Architecture

- Have some architecture in mind that how exactly you're going to code.
- Discuss what exactly pipeline is going to look like.
- Identify what steps will be there in code.
- Accordingly set up the project, folders, codes.

Splitting the RAG Pipeline into Multiple Steps Ingesting a PDF

Step 1 is basically where I'm ingesting a PDF.

- Ingesting a PDF.
- Extract the text out of it.
- Write code to extract text out of it.
- Take any PDF, any path.
- Simply run a code.
- Extract its text output.

Chunking and Page Tracking

• Step 2 will be that I will take this big chunk, the text.

- Once you have done the chunking the text.
- You will basically store it.
- Suppose user asked a question.
- We also want to provide some citation.
- Provide where this chunk is coming from.
- Include page tracking.
- — This chunk is coming from which particular page.

Creating Pinecone Index

• Step 3 .

- Before embeddings, we need to create a pine cone index.
- First of all create a pine cone index.
- Then only you can convert to embedding and store it

Creating Embeddings and Storing in Pinecone

• Step 4.

- Take these chunks.
- Create embeddings.
- Create embeddings and upsert them in pine cone.
- Convert to embedding.
- Store it.

Semantic Search with User Question

• Step 5.

- Semantic search.
- User question.
- Perform semantic search.

Reranking the Results

• Step 6.

- Rerank the results.
- Improve the retrieved results.

Generating the Answer

- **Step 7.**
- Generate answer.
- Use the retrieved and reranked results.

Creating API Endpoints and Testing

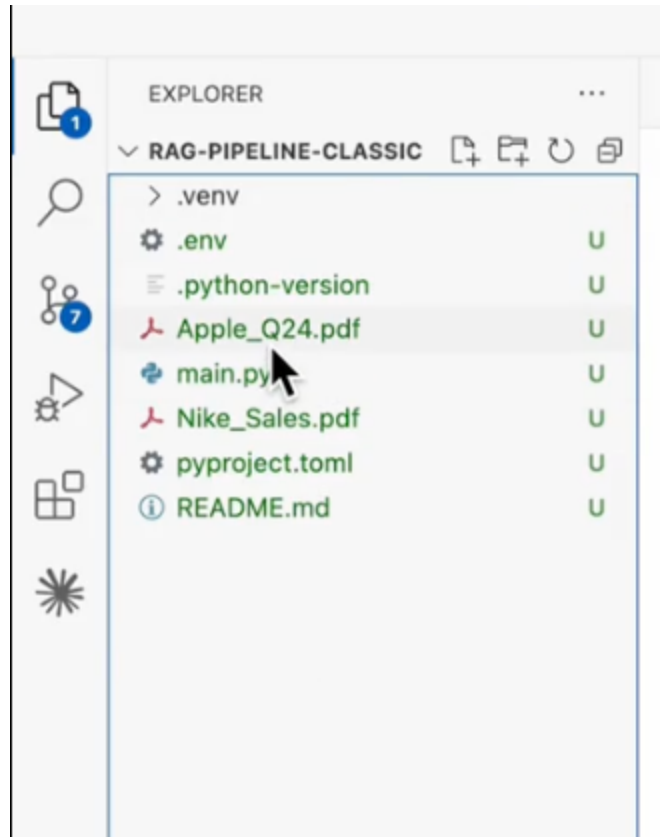
- **Step 8.**
- Create API endpoints.
- Test them.

Complete Coding Architecture

- This is my whole project.
- This is my whole coding architecture.
- These are the steps I need to follow.

Project Root Folder Setup

- This is my root folder.
- A general good coding practice is that.



Documents Used for the Project

Apple Sales Report PDF

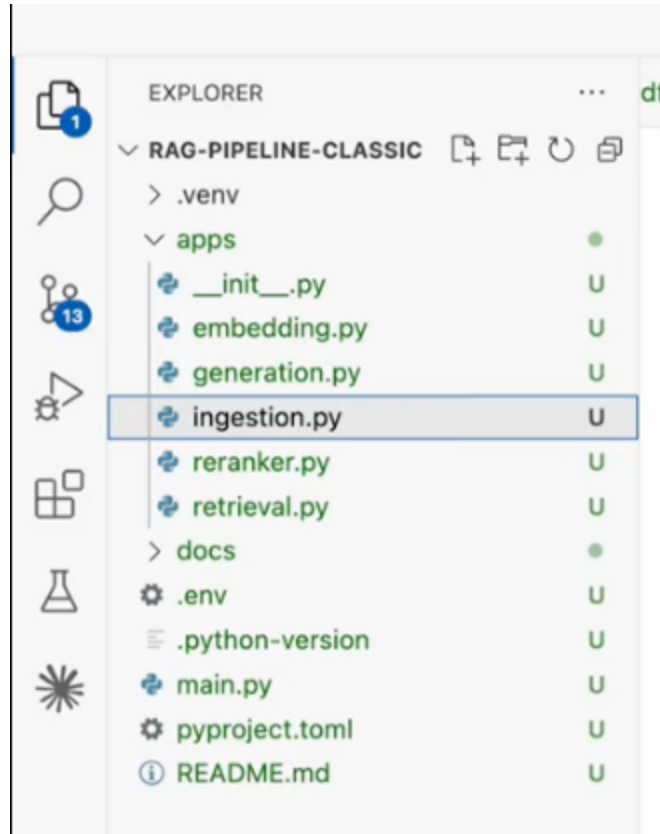
- Apple sales report.
- Short PDF. Four page only.
- Contains a lot of information.
- Sales information of various categories.

Nike Sales Document

- 100 page PDF.
- Contains a lot of information related to 9.
- Some static information.
- Executive officers details.
- Financial information present in this report.
- This PDF is pretty good to understand what exactly we want to interpret out of it.

Creating Documents Folder

- Create a folder.
- Call it docs.



Project Folder Structure and Code Module Design

Creating Root-Level Application Folder

- Create a folder SRC.
- Or create apps.
- Both can be used interchangeably.
- Call it app.
- Inside app, have different levels.

- Have different type of snippets for code.

Initializing the Package

- Before starting, initiate this package.
- Create **init.py**.

The app Folder Is the Package

- The package is the app folder.
- When you create **init.py** inside app, that **app** folder becomes a package.

What That Means

- **app** is now treated as a Python package.
- Files inside it become modules.
 - ingestion.py
 - embedding.py
 - retrieval.py
 - reranker.py
 - generation.py

Relationship Between Package and Modules

- Folder = package (because of **init.py**)
- .py files inside = modules

Example

If your structure is:

project_root

- app
 - **init.py**
 - ingestion.py

Then:

- app is the package.
- ingestion is a module inside the app package.

Why This Matters

- Now you can write:

from app.ingestion import some_function

- Python understands app because **init.py** makes it a package.
- Without **init.py**, structured imports may not work properly in many setups.

So in your architecture, app is the package.

- This will create this module.

Ingestion Module

- First file inside app is ingestion.
- Create ingestion.py.
- Through this code, ingest.

Embedding Module

- After ingestion, next step is embedding.
- Create embedding.py.
- This file is used to create embedding.

Retrieval Module

- Once embedding is done.
- Next step should be retrieval.
- Create retrieval.py.

Reranker Module

- After retrieval.
- Create reranker.py.
- This handles reranking.

Generation Module

- Once reranker is done.
- Create generation.py.
- This handles generation.

Final app Folder Structure

- app
 - **init.py**
 - ingestion.py
 - embedding.py
 - retrieval.py
 - reranker.py
 - generation.py

Explanation

- **init.py** creates the module structure.
- ingestion.py handles ingest.
- embedding.py handles embedding creation.
- retrieval.py handles retrieval.
- reranker.py handles reranking.
- generation.py handles generation.